applied sciences

MDPI

Article

Representation-Centric Approach for Android Malware Classification: Interpretability-Driven Feature Engineering on Function Call Graphs

Gyumin Kim, Dongmin Yoon, NaeJoung Kwak * and ByoungYup Lee *

Department of Information Security, Pai Chai University, Daejeon 35345, Republic of Korea; rbals0520123@naver.com (G.K.); nky7942@naver.com (D.Y.)
* Correspondence: knj0125@pcu.ac.kr (N.K.); bylee@pcu.ac.kr (B.L.)

Abstract

The existing research on Android malware detection using graph neural networks (GNNs) has largely focused on architectural improvements, while input node feature representations have received less systematic attention. This study adopts a representation-centric approach to enhance function call graph (FCG)-based malware classification through interpretability-driven feature engineering. We propose a dual-level structural feature framework integrating local topological patterns with global graph-level properties. The initial feature set comprises 13 dimensions: five local degree profile (LDP) features and eight global structural features capturing community structure, execution flow, and connectivity patterns. To mitigate the curse of dimensionality, we apply an interpretability-driven selection using integrated gradients (IG), gradient-weighted class activation mapping (GradCAM), and Shapley additive explanations (SHAP), yielding an optimized seven-dimensional subset. Experiments on the MalNet-Tiny benchmark demonstrate that the proposed approach achieves 94.47 $\pm$ 0.25% accuracy with jumping knowledge GraphSAGE (JK-GraphSAGE), improving the LDP-only baseline by 0.32 percentage points while reducing feature dimensionality by 46%. The selected features exhibit consistent importance across four GNN architectures and multiple message-passing layers, demonstrating model-agnostic effectiveness. The results reveal that aggregation mechanisms critically influence feature utility, highlighting the necessity of interpretability-guided design for robust malware detection. This work provides a systematic methodology for feature engineering in graph-based security applications.

Keywords: android malware detection; graph neural networks; function call graph; explainable AI; interpretability-driven feature selection

Check for updates

Academic Editors: Jian Xu and Ruijin Wang

Received: 13 February 2026
Revised: 3 March 2026
Accepted: 9 March 2026
Published: 11 March 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Android is the most widely used mobile operating system worldwide, and accordingly, Android malware continues to proliferate [1]. While traditional signature-based malware detection techniques are effective against known threats, they face fundamental limitations when confronting obfuscation techniques, packing methods, or novel malware variants. Static analysis methods offer high efficiency but are vulnerable to code transformations, whereas dynamic analysis methods face challenges in large-scale deployment due to high computational costs and limited execution coverage [2,3]. As an alternative to overcome these limitations, recent approaches that model programs as graph structures and apply graph neural networks (GNNs) have garnered significant attention.

In particular, function call graphs (FCGs) can effectively represent the structural characteristics of programs by clearly expressing inter-procedural call relationships, and various GNN architectures (GCN [4], GraphSAGE [5], GIN [6], JK-GraphSAGE [7], etc.) have been applied to FCG-based Android malware detection [8,9]. However, the existing studies have primarily focused on comparing GNN architecture performance or structural improvements with a relatively limited systematic analysis of how the design and quality of input node feature representations affect detection performance.

Most existing FCG-based approaches utilize simple local structural features, such as the local degree profile (LDP) as node features [9–11]. While the LDP offers advantages of low computational cost and simple implementation, it reflects only limited neighborhood ranges and fails to adequately capture critical information such as global graph structure or execution flow. This limitation of feature representation suggests that the primary bottleneck in detection performance may lie in the input features themselves, despite the sufficient expressive power of GNNs. This raises the necessity of shifting from the traditional model-centric approach to a representation-centric perspective that focuses on the input representation itself [12,13].

Therefore, this paper proposes a dual-level structural feature framework that integrates local topological information with global graph-level structural characteristics. The proposed framework enhances the information content and expressiveness of node representations by extending the existing LDP features to include global features that capture community structure, connectivity, and execution flow. Furthermore, to address the inefficiency and noise issues arising from increased feature dimensionality, we apply an interpretability-driven feature selection method combining integrated gradients [14], Grad-CAM [15], and SHAP [16] to derive a more compact yet discriminative feature set.

The main contributions of this paper can be summarized as follows:

- We propose a dual-level structural feature framework that integrates local topological information with global graph structure information, overcoming the limitations of input feature representation in FCG-based Android malware detection.
- Through an interpretability-driven feature selection method combining integrated gradients, GradCAM, and SHAP, we derive an optimal feature set that effectively reduces dimensionality while maintaining performance.
- We experimentally validate the model-agnostic effectiveness of features across various GNN architectures (GCN, GIN, GraphSAGE, JK-GraphSAGE) and throughout message-passing layers.
- By analyzing the differences in feature effectiveness according to aggregation mechanisms, we demonstrate the importance of interpretability-driven feature design for robust malware detection.

The remainder of this paper is organized as follows. Section 2 reviews related work and analyzes the limitations of the existing LDP-based approaches. Section 3 describes in detail the proposed dual-level structural feature framework and interpretability-driven feature selection method. Section 4 presents the experimental setup and performance evaluation results, analyzing the generalization performance across various GNN architectures and aggregation mechanisms. Finally, Section 5 presents the conclusions of this study and future research directions.

2. Related Work

2.1. Overview of Android Malware Detection Techniques

Android malware classification research has evolved through static analysis (permissions, API calls), dynamic analysis (system calls, network traffic), and hybrid analysis approaches. Static analysis is efficient but vulnerable to obfuscation, while dynamic analy-

sis faces high sandbox costs and limited code coverage. Hybrid analysis methods (Hybrid, C2Miner) have improved detection rates but face limitations in data fusion complexity and insufficient reflection of global structure [17,18].

Recently, graph-based approaches that represent program structure as graphs and learn semantic relationships have attracted significant attention. Control flow graphs (CFGs) represent execution paths but do not directly express inter-procedural call relationships and tend to become large-scale. By contrast, function call graphs (FCGs) effectively capture the global structure (inter-procedural relations) of programs by representing inter-function call relationships as nodes and edges, which are relatively robust to obfuscation and provide function-level contextual information [19,20].

Furthermore, beyond simply improving detection accuracy, research in 2024 and 2025 has focused on enhancing the reliability of GNN models. In particular, explainable GNN research applying techniques such as dual subgraph matching to interpret the 'black-box' decision-making process of GNNs [17], and adversarial robustness research enhancing defense against subtle graph perturbation attacks [18] have been actively conducted. This demonstrates that interpretability and security are essential, along with high performance, for deploying malware detection systems in real-world environments.

MalNet, a representative FCG dataset, contains 1.2 million FCGs from 696 families, with each graph having an average of 15,000 nodes and 35,000 edges [8]. MalNet-Tiny provides 5000 FCGs (maximum 5000 nodes) with 1000 samples each from five classes (AdDisplay, Adware, Benign, Downloader, Trojan), serving as a standard benchmark for FCG-based models [8,9].

2.2. Graph Neural Network-Based Malware Detection

Graph neural networks (GNNs) are deep learning models that learn node embeddings by aggregating information from neighboring nodes and have been applied to malware detection. Early GCNs (graph convolutional networks) learned node embeddings based on spectral filtering but had high computational complexity and constraints in processing large-scale graphs.

To address this, GraphSAGE introduced sampling-based aggregation to significantly improve scalability. GraphSAGE samples a fixed number of neighbors rather than all neighbors, reducing computational cost while enabling inductive learning. A GIN (graph isomorphism network) achieved graph representation power at the level of the Weisfeiler–Lehman test by ensuring injectivity [6].

However, general GNNs suffer from the over-smoothing problem as depth increases, reducing node discrimination, and fail to sufficiently learn global structural information due to the locality of message passing. To address this, the jumping knowledge (JK) technique was proposed [7]. JK generates final node representations by concatenating outputs from all layers from 1-hop to K-hops rather than discarding them, integrating multi-scale information and alleviating over-smoothing.

JK-GraphSAGE is currently the state-of-the-art model for FCG-based malware detection, combining GraphSAGE's sampling efficiency with JK's multi-scale integration to achieve 94.40% accuracy on MalNet-Tiny [9]. In our reproduction, JK-GraphSAGE with LDP_5D achieves 94.15%, with the slight difference attributable to independent data splitting and fixed random seeds applied in this study. However, Lo et al.'s work uses only the local degree profile (LDP) as node features, failing to sufficiently reflect global structure beyond 2-hops or execution flow information.

2.2.1. Local Degree Profile (LDP)

The core of the existing function call graph (FCG) research lies in the embedding process that converts graphs into numerical feature vectors. One of the most widely adopted methods for this purpose is the local degree profile (LDP). The LDP captures the local structural characteristics of a node by summarizing its one-hop neighborhood using five degree-based statistics:

$$LDP(v) = [deg(v), min_{nb}(v), max_{nb}(v), mean_{nb}(v), std_{nb}(v)] \quad (1)$$

where $deg(v)$ is the degree of node v (i.e., the number of directly connected nodes), $min_{nb}(v)$ and $max_{nb}(v)$ represent the minimum and maximum degrees among v's one-hop neighbors, respectively, and $mean_{nb}(v)$ and $std_{nb}(v)$ denote the mean and standard deviation of the degrees of v's one-hop neighbors.

The LDP is computationally efficient, with linear time complexity $O(|V|)$, and provides an intuitive representation of local call patterns in function call graphs. As a result, it has become a standard node feature representation in numerous prior studies, including the work of Lo et al. For example, a high value of $deg(v)$ often indicates important functions, such as dispatchers or controllers, that interact with many other functions. By contrast, a high $std_{nb}(v)$ reflects a heterogeneous neighborhood structure, suggesting the coexistence of highly connected hub functions and sparsely connected peripheral functions within the local region of the graph [9–11,21].

2.3. Limitations of LDP-Based FCG Representations

While the LDP and existing GNN approaches have shown high performance in FCG-based malware detection, they have three fundamental limitations:

Limitation 1: Restricted Receptive Field (Locality Issue)

The LDP considers only direct neighbors (1-hop) and fails to capture information from neighbors beyond 2-hops. Malware uses multi-stage calls for detection evasion. For example, in a structure where Entry Point reaches MaliciousPayload through Dispatcher and Obfuscator, Entry Point and MaliciousPayload are 3-hops apart, making their relationship undetectable with the LDP alone.

Additionally, core malware functions (data exfiltration, remote control) often form dense function groups (communities), but such community structures formed through indirect connections beyond 2-hops cannot be detected by the LDP, which considers only 1-hop [22,23].

Limitation 2: Absence of Global Structural Information (Lack of Global Structure)

The LDP computes only node-centric local features and fails to reflect the overall graph structure. Global information important for malware classification includes the following:

- Community density: malicious payloads concentrated in specific communities;
- SCC (strongly connected component) size: cyclic structures for persistent malicious behavior;
- Graph centrality: global position of core control functions.

For example, even if two functions A and B have an identical LDP, if A is positioned at the graph center with many indirect connections while B is isolated at the periphery, their roles and importance differ significantly. The LDP does not include such global positional information [12,13].

Limitation 3: Static Structure Only

The LDP is a static feature representing only inter-function connection structure, not including dynamic information such as function call order or execution stages during program execution.

Android app execution flow has a hierarchical structure as follows:

- Layer 0: Entry points (onCreate, onStart, etc.)—directly called by system;
- Layer 1: High-level controllers—main control logic;
- Layer 2: Business logic—core functionality implementation;
- Layer 3: Utility functions—auxiliary functions.

Malware tends to hide malicious behavior in deep call stages (Layer 2–3) for detection evasion, while benign apps tend to position main functionality near entry points (Layer 0–1). Call depth is critical discriminative information that the LDP fails to capture. Furthermore, whether a node is an entry point is important, but the LDP simply computes node degree and cannot distinguish between entry points and general functions [24,25].

These limitations motivate the design of richer structural features that can integratively reflect both local and global structure, which are described in the next section.

3. Dual-Level Structural Feature—Aware GNN Framework

3.1. Overview of Proposed Framework

As discussed in Section 2.3., most existing FCG-based Android malware detection studies primarily focus on advancing the graph neural network (GNN) architecture, while the design of input node features remains largely dependent on the local degree profile (LDP). Such representations emphasize local neighborhood statistics but provide limited information about the global structural properties and execution flow characteristics of function call graphs.

To address this limitation, this study presents a structure-aware framework that explicitly incorporates both local-level and global-level structural information into node feature representations. Instead of modifying or extending the GNN architecture, the framework focuses on enhancing the quality of structural features extracted from function call graphs. This design choice follows a representation-centric perspective, aiming to improve the expressiveness of input features in a manner that is independent of specific GNN models.

An overview of the entire framework is illustrated in Figure 1. The framework consists of three sequential stages: dual-level structural feature extraction, interpretability-based feature selection, and graph-level classification.

In the first stage, dual-level structural features are extracted from function call graphs. Local-level features are derived based on the local degree profile statistics to summarize neighborhood connectivity patterns, while global-level features are designed to capture graph-wide properties, such as connectivity patterns, community-related characteristics, and execution flow structure. Together, these features form an enriched structural representation for each node.

In the second stage, an interpretability-driven feature selection procedure is applied. Since different structural features may contribute unequally to classification, multiple attribution methods—including integrated gradients, Grad-CAM, and SHAP—are jointly employed to analyze feature relevance. Based on this analysis, a compact subset of informative structural features is selected for subsequent classification.

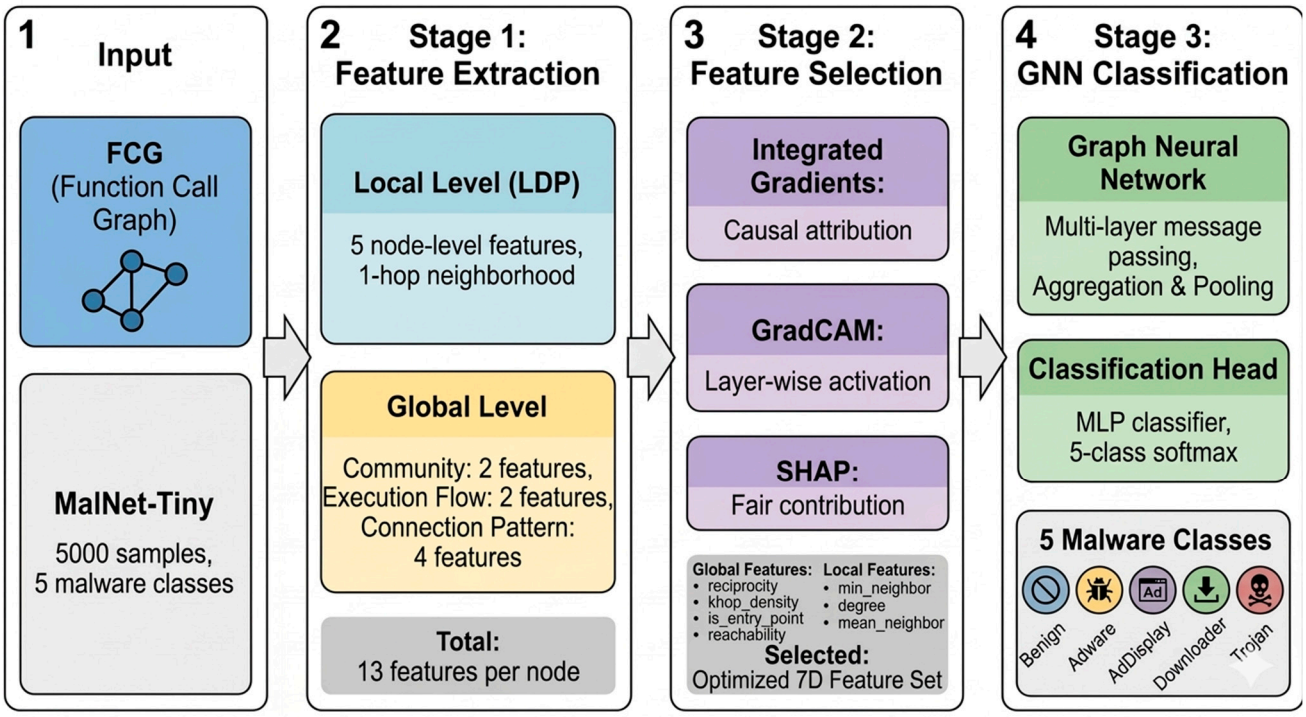


Figure 1. Proposed System Architecture.

In the final stage, the selected feature set is used as input to a JK-GraphSAGE-based graph classification model, which aggregates multi-level node representations to produce graph-level predictions. This stage evaluates how the selected structural features interact with GNN-based message passing and aggregation mechanisms.

The detailed definitions of the dual-level structural features are provided in Section 3.2, while Section 3.3 describes the interpretability-based feature selection methodology. The actual selection results and performance evaluation are presented in Section 4.

3.2. Dual-Level Structural Feature Extraction

All FCGs in this study are treated as directed graphs, preserving the caller→callee directionality of function invocations. For each node v, $in_deg(v)$ denotes the number of incoming edges (i.e., functions that call v), and $out_deg(v)$ denotes the number of outgoing edges (i.e., functions called by v). This directed structure is maintained throughout all feature computations unless otherwise specified.

At the local level, we use the local degree profile (LDP), which summarizes each node's 1-hop neighborhood structure. As defined in Section 2.2.1 (Equation (1)), the LDP is an efficient local feature validated in the existing research. We maintain the LDP in this study for two key reasons, as follows: baseline and fair comparisons and local patter efficiency.

Baseline and fair comparisons. Since the LDP is the standard feature in prior studies, including Lo et al. [9], maintaining it allows us to clearly validate that performance improvements stem from the addition of global level features.

Local pattern efficiency. The LDP efficiently captures local call patterns of 1-hop neighborhoods. High degree indicates dispatchers/controllers, while high standard deviation represents heterogeneous neighborhood structure, enabling intuitive interpretation [9–11,21].

While the LDP efficiently captures 1-hop neighborhood information, as discussed in Section 2.3, it fails to include global structure beyond 2-hops and execution flow information. To complement this, we add global level features in the following subsection.

3.2.1. Global Level Structural Features

At the global level, we add eight features that reflect the overall structural relationships of the graph that the LDP fails to capture. These directly address the three limitations in Section 2.3 from the following three perspectives: community structure, execution flow, and connectivity patterns.

Community Structure Features (2 Features)

Addressing Limitation 1 from Section 2.3: Restricted Receptive Field

The LDP considers only 1-hop and cannot detect community structures formed through indirect connections beyond 2-hops. Since core malware functions (data exfiltration, remote control) tend to form dense function groups, community-level structural information is necessary.

G1. K-hop Subgraph Density (khop_density)

Equation (2) measures how tightly functions within a k-hop of a node call each other.

$$khop_density(v) = \frac{|E_k(v)|}{|V_k(v)| \cdot (|V_k(v)| - 1)} \tag{2}$$

where $V_k(v)$ is the set of all nodes within shortest path distance k from node v, $E_k(v)$ is the set of edges within $V_k(v)$, and $k = 2$ in this study.

Intuition: High khop_density indicates the existence of a dense function group (community) around v. This explicitly captures 2-hop relationships that the LDP misses.

Relevance to malware detection: Malware exhibits patterns where functions performing malicious payloads (data collection, encryption, transmission) are concentrated in specific communities. Benign apps have lower density due to low inter-module dependencies [22,23].

G2. Strongly Connected Component Size (scc_size)

Equation (3) measures the size of the strongly connected component (SCC) to which a node belongs.

$$\text{scc_size}(v) = \frac{|\text{SCC}(v)|}{|V|} \tag{3}$$

where $SCC(v)$ is the strongly connected component containing node v, $|SCC(v)|$ represents the number of nodes in the SCC, and $|V|$ is the total number of nodes in the graph.

Intuition: Large SCC indicates cyclic call structure, meaning functions are mutually recursively connected.

Relevance to malware detection: Malware forms cyclic structures for persistent behavior (command reception loops, state monitoring). A large SCC can be an indicator of such persistent malicious behavior [26].

Execution Flow Features (2 Features)

Addressing Limitation 3 from Section 2.3: Static Structure Only

The LDP represents only the inter-function connection structure and does not include dynamic information, such as function call order or execution stages during program execution. Since malware hides malicious behavior in deep call stages for detection evasion, execution flow information is necessary.

G3. Is Entry Point (is_entry_point)

Equation (4) measures the binary feature indicating whether a node is an entry point directly called by the system.

$$is_entry_point(v) = \begin{matrix} 1 & if\ v \in E \\ 0 & otherwise \end{matrix} \tag{4}$$

In this study, entry points are identified as nodes with in-degree zero in the directed FCG—i.e., functions not called by any other function within the graph. If no zero-in-degree node exists (e.g., in graphs with only cyclic structures), nodes with the minimum in-degree are used as fallback entry points. Android lifecycle methods, such as onCreate and onStart, typically satisfy this criterion, though the detection is purely structure-based rather than name-based.

Intuition: Entry points are the starting points of program execution, corresponding to Android lifecycle functions or event handlers.

Relevance to malware detection: Benign apps position main business logic near entry points, but malware executes malicious payloads through multiple stages from entry points for detection evasion. Entry point information provides execution context of functions.

G4. Call Depth (call_depth)

Equation (5) measures the shortest call depth from entry points to the node.

$$depth_raw(v) = min\{dist(e, v) | e \in E\} \tag{5}$$

$$call_depth(v) = \frac{min_{e \in E} dist(e, v)}{max_{u \in V}(min_{e \in E} dist(e, u))} \tag{6}$$

where $dist(e, v)$ represents the shortest path length from entry point e to node v, and the denominator in Equation (6) normalizes the maximum depth of the graph.

Intuition: Call depth indicates how deep a function is positioned in the execution flow. Low values mean near entry points, high values mean deep call stages.

Relevance to malware detection: Malware hides malicious behavior in deep call stages (Layer 2–3) for detection evasion, while benign apps position main functionality near entry points (Layer 0–1). Call depth captures these execution hierarchy patterns.

Call depth is computed via multi-source BFS on the directed graph, following edge direction (caller→callee) simultaneously from all entry points. Nodes unreachable from any entry point are assigned the maximum observed depth in the graph prior to normalization by Equation (6).

Connectivity Pattern Features (4 Features)

Addressing Limitation 2 from Section 2.3: Absence of Global Structural Information

The LDP computes only node-centric local features and fails to reflect the position or role of nodes in the entire graph. Even if two nodes have an identical LDP, their roles can differ according to global position, so global connectivity pattern information is necessary.

G5. Fan-out Ratio (fan_out_ratio)

Equation (7) measures the ratio of outgoing connections of a node.

$$fan_out_ratio(v) = \frac{out_deg(v)}{in_deg(v) + out_deg(v)} \tag{7}$$

where $out_deg(v)$ is the number of outgoing edges from v (functions called by v) and $in_deg(v)$ is the number of incoming edges to v (functions calling v).

Intuition: Values close to 1 indicate v is primarily a control function calling other functions, while values close to 0 indicate a callee function primarily being called.

Relevance to malware detection: Malware dispatchers or controllers have a high fan-out, while utility functions have a low fan-out. This indicates the control flow role of functions.

G6. Reciprocity (reciprocity)

Equation (8) measures the ratio of the bidirectional call relationships of a node.

$$reciprocity(v) = \frac{|Succ(v) \cap Pred(v)|}{|Succ(v) \cup Pred(v)|} \tag{8}$$

where $Succ(v)$ is the successor set of v (functions called by v) and $Pred(v)$ is the predecessor set of v (functions calling v). The numerator represents the number of neighbors with bidirectional edges with v, while the denominator represents the total number of neighbors of v.

Intuition: High reciprocity means v is connected to functions that mutually recursively call each other.

Relevance to malware detection: Malware tends to form mutual call structures (command reception ↔ command execution) for persistent behavior, showing higher reciprocity than benign apps.

G7. K-hop Reachability (reachability)

Equation (9) measures the ratio of nodes reachable within k-hops from a node.

$$reachability(v) = \frac{|\{u \in V \mid dist(v,u) \leq k\}| - 1}{|V|} \tag{9}$$

where $dist(v,u)$ represents the shortest path length from node v to u, $k = 2$ in this study, and the subtraction of 1 excludes the node itself from the reachable set.

Intuition: High reachability means v is positioned at the graph center and is indirectly connected to many functions.

Relevance to malware detection: Core control functions of malware are connected to the entire malicious module and have high reachability, while independent modules of benign apps have low values.

G8. Clustering Asymmetry (clustering_asymmetry)

Equations (10) and (11) measure the asymmetry of incoming and outgoing clustering coefficients of a node.

$$C_{in}(v) = \frac{|\{(u,w) \in E \mid u,w \in Pred(v)\}|}{|Pred(v)|(|Pred(v)|-1)} \tag{10}$$

$$C_{out}(v) = \frac{|\{(u,w) \in E \mid u,w \in Succ(v)\}|}{|Succ(v)|(|Succ(v)|-1)} \tag{11}$$

$$asymmetry(v) = |C_{in}(v) - C_{out}(v)| \tag{12}$$

where $Pred(v)$ is the predecessor set of v (functions calling v) and $Succ(v)$ is the successor set of v (functions called by v). $C_{in}(v)$ measures the connection density among functions calling v, while $C_{out}(v)$ measures the connection density among functions called by v.

Intuition: High asymmetry means v's incoming neighbors and outgoing neighbors have different structural patterns.

Relevance to malware detection: Malware shows asymmetric patterns of being called from multiple entry points (low C_{in}) but concentrating calls to specific malicious modules (high C_{out}).

3.2.2. Feature Integration and Normalization

For each node v, we concatenate five local level and eight global level features to generate a 13-dimensional feature vector.

$$X_v = [LDP(v) \parallel Global(v)] = [L_1, L_2, L_3, L_4, L_5, G_1, G_2, G_3, G_4, G_5, G_6, G_7, G_8\,] \tag{13}$$

For the entire graph, we construct the feature vectors of all nodes in matrix form.

$$X \in \mathbb{R}^{|V| \times 13} \tag{14}$$

The complete dual-level structural feature set is summarized in Table 1.

Table 1. Composition of Dual-Level Structural Features.

Level	Category	Feature	Description	Range
Local (LDP)		degree	Node degree	$[0, max]$
		min_neighbor	Minimum neighbor degree	$[0, max]$
		max_neighbor	Maximum neighbor degree	$[0, max]$
		mean_neighbor	Mean neighbor degree	$[0, max]$
		std_neighbor	Std of neighbor degree	$[0, \infty)$
Global	Community Structure	khop_density	K-hop subgraph density	$[0, 1]$
		scc_size	SCC size	$[0, 1]$
	Execution Flow	is_entry_point	Entry point indicator	$[0, 1]$
		call_depth	Call depth from nearest entry point	$[0, 1]$
	Connection	fan_out_ratio	Ratio of out going to total calls	$[0, 1]$
		reciprocity	Fraction of bidirectional call pairs	$[0, 1]$
	Pattern	reachability	Fraction of nodes reachable from a node	$[0, 1]$
		clustering_asym	Asymmetry between in/out clustering coefficients	$[0, 1]$

Normalization: Since the features have different ranges and distributions, we use QuantileTransformer [27] to transform each feature to a uniform distribution in the $[0, 1]$ range to enhance learning stability and convergence speed. QuantileTransformer computes the cumulative distribution function (CDF) of each feature to map data to a uniform distribution, is robust to outliers, and normalizes feature distributions through nonlinear transformation. Quantiles computed on the train dataset are applied identically to validation/test datasets to prevent data leakage.

3.3. Interpretability-Based Feature Selection

3.3.1. Necessity of Feature Selection

The 13 features designed in Section 3.2 can theoretically overcome the limitations of the LDP, but not all features contribute equally to actual classification. Some features may degrade performance for the following reasons: curse of dimensionality, noisy features, training instability, and feature interaction.

Curse of dimensionality. As the number of features increases, training data becomes sparse, increasing the risk of overfitting. GNNs in particular can become sensitive to noise with high-dimensional inputs.

Noisy features: Low discriminative power or redundant information with strong correlation to other features degrades the model's generalization performance.

Training instability: If some features provide inconsistent signals during the learning process, model convergence becomes unstable and result reproducibility decreases.

Feature interaction: When features interact in complex ways, simply adding features may not improve performance.

Traditional feature selection methods have the following limitations. Filter methods (correlation-based) do not consider feature interactions, wrapper methods (performance-based search) have exponentially increasing computational costs, and embedded methods (selection during model training) measure importance from only a single perspective.

This study adopts an interpretability-based approach that analyzes the decision-making process of trained GNN models to identify truly informative features. The key advantages of this approach are as follows:

- Model-centric evaluation: Reflects features actually utilized by the trained classifier;
- Feature interaction awareness: Captures how features interact during message passing;
- Computational efficiency: All feature importance is computed in a single training run;
- Multi-perspective validation: Combines results from multiple XAI techniques for robust selection.

3.3.2. Interpretability Methodology

This study uses three complementary XAI techniques to evaluate feature importance from multiple perspectives. Each technique measures feature contribution from different theoretical perspectives as follows:

- Integrated gradients (IG): Analyzes input–output causal paths from an information-theoretic perspective;
- GradCAM: Analyzes layer-wise activation patterns based on gradients;
- SHAP: Measures fair contribution based on game theory.

Integrated Gradients (IG)

Integrated gradients is a method that computes the causal contribution between input features and prediction results through an integral path [14]. The key idea is to integrate gradients along the path from a baseline to the actual input, quantifying the degree to which each feature contributes to prediction changes.

$$IG_i(x) = (x_i - x'_i) \times \int_0^1 \frac{\partial F(x' + \alpha(x - x'))}{\partial x_i} d\alpha \tag{15}$$

where x is the actual input feature vector, x' is the baseline (set to zero for all features in this study), F is the model's prediction function, and α is the interpolation coefficient ranging from 0 to 1.

Key advantages: IG provides two theoretical guarantees: completeness and sensitivity. Completeness guarantees that the sum of all feature contributions matches the prediction change, and sensitivity guarantees that if a feature value change affects prediction, it must be assigned non-zero importance.

Implementation details: In this study, we discretize α into 50 steps to approximate the path integral, compute IG for all test samples, and calculate the global importance of each feature as the average of absolute values.

Gradient-Weighted Class Activation Mapping (GradCAM)

GradCAM is a method that measures the activation contribution of each feature to specific class predictions layer-by-layer [15]. Originally developed to visualize the image region importance in CNNs, in this study we apply it to compute the feature-wise importance at each GNN layer.

$$GradCAM^i = \left(\frac{1}{|V|}\right) \Sigma(v \in V) \left| \frac{\partial y_c}{\partial x_i^v} \right| \tag{16}$$

where y_c is the model output (logit) for class c, x_i^v represents the value of input feature i at node v, $|V|$ is the total number of nodes in the graph, and $\frac{\partial y_c}{\partial x_i^v}$ denotes the gradient measuring feature i's effect on class c prediction.

Key idea: GradCAM computes the gradient of each input feature with respect to prediction, quantifying which features contribute strongly to the final classification result. In this study, we average the gradient absolute values of all nodes to calculate the graph-level feature importance.

Layer-wise analysis: A unique advantage of GradCAM is the ability to analyze feature importance layer-by-layer for each GNN layer. This allows verification of whether specific features are important only at specific hop distances or consistently across all layers.

SHapley Additive exPlanations (SHAP)

SHAP is a method that computes model-agnostic fair contribution based on game theory's Shapley value [16]. The key idea is to average the prediction changes when each feature is removed over all possible feature combinations to calculate fair contribution.

Theoretically, the Shapley value is defined as follows:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \left[\frac{|S|!(|F| - |S| - 1)!}{|F|!} \right] [f(S \cup \{i\}) - f(S)] \quad (17)$$

where φ_i is the SHAP value of feature i, F is the entire feature set, S is a subset excluding feature i, and $f(S)$ is the prediction value when using feature set S.

However, this formula requires evaluation over $2^{|F|}$ for all subsets, making computation very expensive. Therefore, in this study we use KernelSHAP approximation to compute this efficiently.

The key advantage of SHAP is that, according to game theory's uniqueness theorem, it is the only feature attribution method that simultaneously satisfies fairness, local accuracy, and consistency. This guarantees that feature contribution is not affected by the input order of features or the presence of other features.

All global features are computed once in the preprocessing stage and cached as fixed node attributes. This ensures that feature extraction does not affect inference latency, as the computational cost is incurred only during offline preprocessing.

3.3.3. Feature Selection Rule

Since individual interpretability techniques follow different assumptions and calculation methods, there is a risk of producing results biased toward specific perspectives. To avoid such bias, this study adopts an ensemble-based selection approach using rank aggregation.

The specific procedure is as follows. First, we compute importance scores for all features through three techniques: integrated gradients (IG), GradCAM, and SHAP. Then, within each technique, we sort, in descending order of importance, the scores and assign ranks to each feature. That is, the feature with the highest contribution is assigned rank 1.

Finally, the integrated importance of each feature is defined as the average of the normalized importance scores assigned by the three techniques. The higher the averaged score, the more consistently it is evaluated as important across all techniques. The selection criterion is to choose the top-k features with the highest average importance. The detailed procedure is formalized in Algorithm 1, and the actual selection results are presented in Section 4.4.3.

Algorithm 1: Interpretability-based Feature Selection

Input Trained JK-GraphSAGE model M with 13D features, Test dataset D_{val}

Output Selected feature subset $F_{selected}$(Optimized_7D)

// Step 1: Compute feature importance scores
$I_{IG} \leftarrow ComputeIntegratedGradients(M, D_{val})$
$I_{GradCAM} \leftarrow ComputeGradCAM(M, D_{val})$
$I_{SHAP} \leftarrow ComputeSHAP(M, D_{val})$

// Step 2: Normalize scores to [0, 1] for each method
for each method $m \in \{IG, GradCAM, SHAP\}$ do
    $I_m \leftarrow (I_m - min(I_m))/(max(I_m) - min(I_m)$
end for

// Step 3: Compute average importance per feature
for each feature $i \in \{1, \ldots, 13\}$ do
    $I_{avg}[i] \leftarrow (I_{IG}[i] + I_{GradCAM}[i] + I_{SHAP}[i])\ /\ 3$
end for

// Step 4: Rank features by average importance
$R \leftarrow SortDescending(I_{avg})$

// Step 5: Select top-7 features
$F_{selected} \leftarrow R[1:7]$

return $F_{selected}$ // Optimized_7D

4. Experimental Results

4.1. Experimental Setup

This section presents the dataset, feature sets, training configuration, and evaluation protocol required to validate the performance of the proposed method.

4.1.1. Dataset

This study uses the MalNet-Tiny dataset. MalNet-Tiny is a standard benchmark for Android malware function call graph (FCG) classification, consisting of 5000 FCGs distributed across five classes (AdDisplay, Adware, Benign, Downloader, Trojan) with 1000 samples each. Each graph is limited to a maximum of 5000 nodes and reflects the function call structure of real Android applications.

The dataset is split into Train/Validation/Test sets with a 70:10:20 ratio, containing 3500, 500, and 1000 samples, respectively. Class ratios are maintained equally across all three splits to ensure balanced representation in each partition.

The detailed dataset composition and split ratios used in the experiments are summarized in Table 2. The dataset is partitioned using stratified sampling (StratifiedShuffleSplit) with a fixed random state of 42, ensuring identical class distributions and identical train/validation/test splits across all feature configurations and model architectures. In this study, we fix the data split and compare the performance differences according to feature configurations and model architectures to fairly analyze the effectiveness of the proposed feature representation.

Table 2. MalNet-Tiny Dataset Composition.

Class	Train	Validation	Test	Total
AdDisplay	700	100	200	1000
Adware	700	100	200	1000
Benign	700	100	200	1000
Downloader	700	100	200	1000
Trojan	700	100	200	1000
Total	3500	500	1000	5000

4.1.2. Compared Feature Sets

To validate the effectiveness of the proposed method, we compare three feature sets:

1. LDP_5D (Baseline, five-dimensional): Uses only the local degree profile features, identical to the previous work in [9]:
   - Features: degree, min_neighbor, max_neighbor, mean_neighbor, std_neighbor
   - Purpose: Measure baseline performance using only local-level features.
2. Full_13D (Proposed Dual-Level, 13-dimensional): Includes the complete dual-level structural features designed in Section 3.2:
   - Features: five LDP features + eight Global Degree Profile features (two community + two execution flow + four connection pattern)
   - Purpose: Validate the effectiveness of global-level features.
3. Optimized_7D (Interpretability-based, seven-dimensional): The optimal feature set derived from Full_13D through the interpretability-based feature selection algorithm presented in Section 3.3.
   - Features: three local degree profile (degree, min_neighbor, mean_neighbor) + four global degree profile (khop_density, is_entry_point, reachability, reciprocity)
   - Purpose: Evaluate simultaneous achievement of dimensionality reduction and performance improvement.

4.1.3. Training Configuration

All experiments are conducted with identical training settings on the MalNet-Tiny dataset as follows:

- Optimizer: Adam with learning rate lr = 0.001
- Hidden dimension: 128
- Number of layers: 6
- Dropout rate: 0.2
- Aggregation (JK-GraphSAGE, GraphSAGE): max
- Aggregation (GIN): sum
- Aggregation (GCN): mean
- Batch size: 32, maximum 200 epochs, early stopping based on validation loss
- Feature normalization: QuantileTransformer is used to transform each feature to a uniform distribution, mitigating scale differences among feature distributions. The transformation learned on the training set is applied identically to validation/test sets to prevent data leakage.
- Repeated experiments: Training is performed independently across five random seeds (42, 123, 456, 789, 2024), and the average performance is reported.
- Implementation environment: NVIDIA RTX 4080 Laptop GPU (12GB VRAM), Python 3.10.16, PyTorch 2.3.0, PyTorch Geometric 2.6.1, CUDA 12.1

4.1.4. Evaluation Protocol

The performance evaluation is conducted on the test dataset (1000 samples) using two metrics: accuracy and the macro-averaged F1-score.

(1) Accuracy

Accuracy is defined as the ratio of correctly classified samples among all samples:

$$\text{Accuracy} = \frac{(Number\ of\ correct\ predictions)}{(Total\ samples)} \tag{18}$$

For multi-class classification, Precision, Recall, and F1-score are computed per-class using a one-vs-rest strategy, then averaged using macro-averaging.

(2) Macro-averaged F1-score

The F1-score is calculated for each class and then arithmetically averaged to robustly evaluate the overall performance against class imbalance:

$$F1_macro = \left(\frac{1}{C}\right) \times \Sigma_{\{c=1\}}^{C} F1_c \tag{19}$$

where C is the number of classes (5 in this study) and $F1_c$ is the F1-score for class c, computed as $F1_c = 2 \times \frac{(Precision_c \times Recall_c)}{(Precision_c + Recall_c)}$. Here, $Precision_c = \frac{TP_c}{(TP_c + FP_c)}$ and $Recall_c = \frac{TP_c}{(TP_c + FN_c)}$, where TP_c, FP_c, and FN_c denote true positives, false positives, and false negatives for class c, respectively.

Each experiment is independently repeated across five different random seeds (42, 123, 456, 789, 2024), and the results are reported as mean ± standard deviation (mean ± std). While minor variations may occur due to non-deterministic factors, trend comparisons are possible through repeated experimental averages.

4.2. LDP-Based GNN Architecture Comparison

To select a base architecture for validating the effectiveness of the proposed method, we first compare the performance of four representative GNN models (GCN, GIN, GraphSAGE, JK-GraphSAGE) using only LDP_5D. Table 3 summarizes the test accuracy, F1-score, and standard deviation for each architecture.

Table 3. GNN Architecture Performance Comparison Based on LDP_5D.

Architecture	Test Acc (%)	Test F1	Std	Aggregation	Key Feature
GCN	89.85	0.8981	±0.42	Mean	Spectral filtering
GIN	93.13	0.9314	±0.49	Sum	Injective aggregation
GraphSAGE	93.73	0.9376	±0.61	Mean	Sampling-based
JK-GraphSAGE	94.15	0.9417	±0.11	Mean + JK	Multi-scale integration

Architecture characteristics: The four architectures exhibit distinct structural properties that affect their suitability for an FCG analysis. GCN's spectral-based aggregation relies on symmetric normalized Laplacian operator, making it inherently less effective for large-scale directed graphs like FCGs. By contrast, GraphSAGE and JK-GraphSAGE employ sampling-based inductive mechanisms that naturally handle variable graph sizes and directed edges, aligning well with MalNet-Tiny's characteristics. Performance also varies across malware classes: a macro-F1 analysis reveals that Benign samples achieve a 91.0% F1-score while Trojan samples show 89.5%, indicating that certain malware types exhibit more complex structural patterns.

JK-GraphSAGE achieves the highest performance at 94.15% accuracy with the lowest variability at 0.11% standard deviation among the four models. This indicates stable

performance in the range of 94.04–94.26% across the five repeated experiments. While GraphSAGE shows high accuracy, it exhibits relatively high variability due to sampling and using only the final layer output. GIN, based on sum aggregation, has high expressiveness but shows increased variance as it reacts sensitively to hub nodes in FCGs with uneven degree distributions. GCN records the lowest performance due to limitations in inductive processing for directional and large-scale graphs, inherent to mean aggregation and spectral filtering characteristics.

Based on these results, this study adopts JK-GraphSAGE as the base architecture for the following reasons: (1) it provides the highest accuracy and lowest standard deviation among the four models, (2) it aligns well with the dual-level (local + global) structural features from Section 3.2 through the jumping knowledge mechanism that integrates multi-scale information from 1–6 hops, and (3) it enables a fair comparison of the proposed feature design effects by using the same architecture as the previous work in [9].

The next section compares the impact of adding global structural features (Full_13D) on each architecture's performance.

4.3. Effect of Dual-Level Structural Features (Full_13D)

This section validates the effectiveness of the proposed global level features by comparing LDP_5D and Full_13D across four GNN architectures. Full_13D is the dual-level structural feature designed in Section 3.2, consisting of five local level (LDP) and eight global level (two community, two execution flow, four connection pattern) features. Table 4 summarizes the test accuracy, standard deviation, and performance change (Δ) with Full_13D introduction for each architecture.

Table 4. LDP_5D vs Full_13D Architecture-wise Performance Comparison.

Architecture	LDP_5D (%)	Full_13D (%)	Δ (%)	Std_5D (%)	Std_13D (%)
GCN	89.85	92.53	+2.68	±0.42	±0.27
GIN	93.13	92.93	-0.20	±0.49	±0.53
GraphSAGE	93.73	94.19	+0.46	±0.61	±0.38
JK-GraphSAGE	94.15	94.43	+0.28	±0.11	±0.44

GCN shows the largest gain with accuracy increasing from 89.85% to 92.53% (+2.68%) and decreased standard deviation (0.42→0.27%) when using Full_13D, indicating that the global level features provide the greatest benefit for this architecture. GCN's spectral-based aggregation is inherently less effective for large-scale directed graphs like FCGs, as it relies on symmetric normalized Laplacian operator that does not naturally handle directed edges. The addition of global structural features compensates for this limitation by explicitly providing graph-wide connectivity information that GCN's local spectral filters cannot capture effectively.

GraphSAGE and JK-GraphSAGE also show consistent performance improvements of +0.46% and +0.28%, respectively, demonstrating that their inductive sampling mechanisms can effectively leverage global structural information difficult to capture with LDP alone. These architectures are well-suited to MalNet-Tiny's variable graph sizes and benefit from features like reciprocity and reachability that provide context beyond the sampled neighborhoods.

By contrast, GIN is the only model among the four that shows performance degradation with Full_13D, declining from 93.13% to 92.93% (−0.20%) with increased standard deviation (0.49→0.53%). This occurs because GIN's sum aggregation method equally amplifies signals from all neighbor information, causing weak features in MalNet-Tiny to be amplified proportionally with strong features, thereby degrading the signal-to-noise ratio. A more detailed analysis of GIN's sensitivity to feature noise reveals the mechanism

behind this degradation. Under sum aggregation, each node accumulates the raw sum of all neighbor features without normalization, meaning that features with low discriminative power contribute equally to the final representation as informative ones. When propagated through six GNN layers, weak signals from features such as scc_size, call_depth, and fan_out_ratio—which exhibit inter-class KL-divergence values below 0.08—accumulate multiplicatively. This progressively dilutes the contribution of highly discriminative features such as reciprocity (KL-divergence: 0.34) and khop_density (KL-divergence: 0.28). By contrast, mean aggregation (used by GCN, GraphSAGE, and JK-GraphSAGE) normalizes by neighborhood size, inherently reducing the relative influence of noise features. This architectural difference explains why GIN is uniquely susceptible to feature noise in the 13D configuration, and why its performance recovers to 93.25% with Optimized_7D after noise features are removed.

In particular, features such as scc_size, call_depth, and fan_out_ratio exhibit overlapping distributions across malware classes in MalNet-Tiny. A quantitative analysis reveals that these features show minimal class separation: mean differences between malware and benign samples are less than 0.15, and KL-divergence values are below 0.08. Under GIN's sum aggregation, these weak discriminative signals accumulate across neighborhoods and compete with informative features like reciprocity (KL-divergence: 0.34) and khop_density (KL-divergence: 0.28), causing the model to weight uninformative patterns and leading to performance degradation.

This result demonstrates that sum aggregation is particularly sensitive to feature quality, and simply adding features without considering their discriminative power can harm performance in noise-amplifying architectures.

Full_13D improves performance over LDP_5D for three out of four architectures (GCN: +2.68%, GraphSAGE: +0.46%, JK-GraphSAGE: +0.28%), effectively addressing the inherent limitations of the LDP, such as its limited receptive field, lack of global structural awareness, and reliance on static local topology. However, the single case of performance degradation in GIN (-0.20%) reveals that incorporating all 13 dimensions does not necessarily yield an optimal configuration.

These architecture-specific results reveal that aggregation mechanisms critically influence feature utility:

- Mean aggregation (GCN, GraphSAGE, JK-GraphSAGE) distributes neighbor information evenly, making it relatively robust to noise. Performance improvements are maintained with Full_13D (+0.28~+2.68%).
- Sum aggregation (GIN) equally amplifies all features, including noise, leading to performance decrease with Full_13D (-0.20%) when weak discriminative features are present.

This analysis demonstrates that feature selection is essential for sum aggregation-based models, motivating the interpretability-driven feature selection approach described in Section 4.4, which identifies and retains only the most discriminative features while removing noise.

The robustness of the observed feature contributions was verified by repeating all experiments with five different random seeds (42, 123, 456, 789, 2024). The performance trends remained consistent across seeds: GCN consistently showed the largest improvements ($+2.68 \pm 0.27\%$), while GIN consistently exhibited slight degradation ($-0.20 \pm 0.53\%$). The relative ranking of architectures by performance gain was stable across all random initializations, confirming that the results are not artifacts of random initialization but reflect genuine architectural characteristics.

4.4. Interpretability-Based Feature Importance Analysis

The results from Section 4.3 show that the global level features are generally beneficial, but not all 13 features contribute equally. This section quantifies the importance of each feature through the interpretability-based analysis designed in Section 3.3 and provides rationale for the Optimized_7D configuration.

Implementation Details: Feature attributions are computed by aggregating node-wise attributions over all nodes in each graph, then averaging across the validation set. For integrated gradients, we use 50 interpolation steps with a zero baseline. For SHAP, we employ KernelSHAP with 100 background samples randomly selected from the training set. GradCAM computes the gradients with respect to node features at each GNN layer output, allowing for a layer-wise importance analysis.

4.4.1. Overall Feature Importance

We apply three XAI techniques—integrated gradients (IG) [14], GradCAM [15], and SHAP [16]—to the JK-GraphSAGE + Full_13D model and normalize the importance scores to the 0–1 range, averaging over the entire test dataset. Table 5 shows the average importance and standard deviation of the 13 features.

Table 5. Interpretability Importance of 13 Features (JK-GraphSAGE + Full_13D).

Rank	Feature	Category	IG	GradCAM	SHAP	Avg	Std
1	reciprocity	Global (Connection)	0.080	0.152	0.192	0.141	0.058
2	min_neighbor	Local (LDP)	0.093	0.112	0.156	0.120	0.032
3	khop_density	Global (Community)	0.108	0.114	0.101	0.108	0.007
4	degree	Local (LDP)	0.102	0.062	0.089	0.085	0.020
5	is_entry_point	Global (Execution)	0.049	0.058	0.140	0.082	0.050
6	reachability	Global (Connection)	0.106	0.071	0.041	0.073	0.033
7	mean_neighbor	Local (LDP)	0.071	0.081	0.051	0.068	0.015
8	std_neighbor	Local (LDP)	0.081	0.073	0.037	0.064	0.023
9	max_neighbor	Local (LDP)	0.066	0.083	0.038	0.062	0.023
10	clustering_asymmetry	Global (Connection)	0.068	0.066	0.048	0.061	0.010
11	fan_out_ratio	Global (Connection)	0.074	0.048	0.047	0.056	0.015
12	call_depth	Global (Execution)	0.053	0.044	0.041	0.046	0.006
13	scc_size	Global (Community)	0.048	0.035	0.019	0.034	0.015

To empirically validate the selection results, we conducted ablation experiments where the bottom-6 features were removed from Full_13D. The validation performance changed by less than $\pm$0.05% (94.43$\rightarrow$94.41%), confirming that these features contribute minimally to classification and can be safely removed without performance loss.

Reciprocity, min_neighbor, and khop_density consistently rank among the top features across all three techniques, achieving the highest average importance scores, indicating that bidirectional call patterns, minimum neighbor degree, and 2-hop subgraph density are the most important structural clues for malware detection. The features degree, is_entry_point, reachability, and mean_neighbor form a group with moderate consistent importance, complementing the overall FCG structure and execution context through basic connectivity, execution entry point identification, global reachability, and average neighbor degree.

Meanwhile, std_neighbor, max_neighbor, clustering_asymmetry, fan_out_ratio, call _depth, and scc_size consistently record low importance (0.034–0.064) across all three techniques. Particularly, scc_size and call_depth show minimal distribution differences between malware and benign apps in MalNet-Tiny, contributing little to the actual classification performance despite their intuitive meaning.

4.4.2. Layer-Wise Importance Consistency

We apply GradCAM to each GNN layer (Layers 1–6) to analyze the layer-wise feature importance. The results are summarized in Table 6.

Table 6. GradCAM Layer-wise Importance for Top Seven Features.

Feature	Layer 1	Layer 2	Layer 3	Layer 4	Layer 5	Layer 6	Variance
reciprocity	0.154	0.151	0.148	0.155	0.165	0.160	2.92×10^{-5}
min_neighbor	0.119	0.119	0.121	0.129	0.139	0.150	1.28×10^{-4}
khop_density	0.115	0.106	0.110	0.103	0.108	0.109	1.32×10^{-5}
degree	0.065	0.063	0.062	0.064	0.063	0.065	1.47×10^{-6}
is_entry_point	0.054	0.055	0.054	0.052	0.051	0.044	1.45×10^{-5}
reachability	0.073	0.075	0.071	0.073	0.073	0.079	6.14×10^{-6}
mean_neighbor	0.079	0.082	0.080	0.073	0.068	0.067	3.42×10^{-5}

The top seven features show consistently low variance across all layers (1.47×10^{-6} to 1.28×10^{-4}), confirming that they are consistently utilized across the multi-scale spectrum from 1-hop to 6-hops. For instance, reciprocity exhibits a non-monotonic pattern, reaching a local minimum at Layer 3 (0.148) before peaking at Layer 5 (0.165), suggesting that its role in capturing bidirectional call structures becomes more pronounced after multi-hop aggregation integrates longer-range dependencies. Similarly, khop_density maintains stable relative importance throughout (Layer 1: 0.115→Layer 6: 0.109) with minor fluctuations, reflecting its consistent contribution to community-level structural encoding across all message-passing scales.

Conversely, some features, like clustering_asymmetry, show high importance in shallow layers but rapidly decrease in deeper layers, exhibiting patterns where they are useful only at specific scales and may act as noise during multi-scale integration.

4.4.3. Selected Feature Set (Optimized_7D)

Based on the analysis results above, this study uses the average importance strategy from Algorithm 1 to select the top 7 features from the 13 features as the final feature set, Optimized_7D. The selection criteria are as follows: (1) consistently high average importance across IG, GradCAM, and SHAP, (2) very low inter-layer variance based on GradCAM indicating stable utilization patterns, and the resulting set comprises three local and four global features, reflecting a balanced representation of both levels. Table 7 summarizes the selected seven features with their average importance, inter-layer variance, and roles.

Table 7. Selected Seven Features (Optimized_7D).

Rank	Feature	Category	Avg Importance	Layer Variance	Selection Reason
1	reciprocity	Global (Connection)	0.141	0.00002	Highest importance + hierarchical consistency
2	min_neighbor	Local (LDP)	0.120	0.00001	Consistently high importance across all layers
3	khop_density	Global (Community)	0.108	0.00002	Representative of community structure
4	degree	Local (LDP)	0.085	0.00000	Basic connectivity information
5	is_entry_point	Global (Execution)	0.082	0.00001	Execution flow information
6	reachability	Global (Connection)	0.073	0.00001	Global reachability
7	mean_neighbor	Local (LDP)	0.068	0.00001	Average neighbor degree

The derived Optimized_7D maintains both the LDP's local information and core global structural information while reducing dimensionality from 13 to 7 by removing

features with low discriminative power. The next section quantitatively analyzes how Optimized_7D improves actual classification performance and stability compared to Full_13D and LDP_5D.

4.4.4. Summary: Validity of Interpretability-Based Selection

The three XAI techniques and layer-wise analysis results show that a small number of core global features (reciprocity, khop_density, reachability, is_entry_point) and some local features (min_neighbor, degree, mean_neighbor) consistently contribute to actual classification decisions. Conversely, scc_size and call_depth showed consistently low importance across all three techniques contrary to theoretical intuition, classifying them as removal candidates. Therefore, Optimized_7D is not an arbitrary dimensionality reduction but an interpretability-based selection result reflecting the model's decision-making mechanism, with its validity to be quantitatively verified in the performance evaluation of the next section.

It is worth noting that scc_size's low discriminative power in MalNet-Tiny does not necessarily contradict its theoretical relevance to malware detection. While large SCCs are conceptually associated with persistent malicious behavior loops, benign applications with complex UI event-driven architectures (e.g., activity lifecycle callbacks, handler loops) also exhibit large SCCs of comparable size. In MalNet-Tiny, the mean scc_size for benign graphs (normalized: 0.031) and malware graphs (normalized: 0.038) differ by less than 0.01, with substantial distributional overlap across all five classes. This suggests that, in the specific context of MalNet-Tiny's category-level classification (AdDisplay, Adware, Benign, Downloader, Trojan), scc_size alone does not provide sufficient class-separating signal. The feature may become more discriminative in datasets with finer-grained family-level labels or in dynamic analysis settings where execution trace SCCs can be measured independently of benign UI loops.

4.5. Performance of Optimized_7D (JK-GraphSAGE)

This section compares the performance of three feature sets (LDP_5D, Full_13D, Optimized_7D) on the JK-GraphSAGE architecture. Table 8 summarizes the test accuracy, F1-score, standard deviation, and minimum–maximum values for each feature set.

Table 8. Performance Comparison by Feature Set on JK-GraphSAGE.

Feature Set	Test Acc (%)	Test F1	Std (%)	Min Acc (%)	Max Acc (%)	Features	Dimension
LDP_5D	94.15	0.9417	±0.11	94.01	94.31	Local only	5
Full_13D	94.43	0.9444	±0.23	94.11	94.71	Local + Global	13
Optimized_7D	94.47	0.9446	±0.25	94.31	94.91	Selected	7

Optimized_7D records higher values than both LDP_5D and Full_13D in both average and minimum accuracy, maintaining or slightly improving performance while achieving approximately 46% dimensionality reduction from 13 to 7 dimensions. Compared to LDP_5D, average and minimum accuracy show improvements of approximately 0.3%, and even compared to Full_13D, it achieves equal or slightly superior results.

Individual seed-wise convergence patterns and Train/Validation/Test performance are organized in Table 9.

Table 9. Five-Seed Individual Results for JK-GraphSAGE + Optimized_7D.

Seed	Epoch	Test Acc (%)	Test F1	Train Acc (%)	Val Acc (%)
42	195	94.41	0.9443	97.89	91.80
123	163	94.41	0.9440	97.46	92.40
456	132	94.31	0.9431	97.06	92.20
789	186	94.90	0.9487	97.60	91.20
2024	89	94.31	0.9428	96.83	92.80
Average	153 ± 43	94.47 ± 0.25	0.9446 ± 0.0020	97.37 ± 0.41	92.08 ± 0.60

Table 9 confirms that the test accuracy is 94.31% or higher for all seeds, exceeding the baseline LDP_5D's minimum value (94.01%) by 0.30% or more. Early stopping epochs average 153 ± 43 epochs, similar to LDP_5D (average 153 epochs), showing that convergence speed is maintained despite the reduction in feature count. Importantly, the gap between training and test accuracy (~3%) remains stable across seeds, and the consistent validation performance suggests that the model does not exhibit severe overfitting.

To examine the consistency of the performance improvement across random initializations, we conducted a paired t-test comparing Optimized_7D and LDP_5D across the five random seeds. The test yields t = 1.76, p = 0.153, which does not reach conventional significance thresholds, as expected given the limited sample size (n = 5). Nevertheless, Optimized_7D achieves equal or higher accuracy than LDP_5D in all five seeds, and the 0.30% mean improvement is consistent in direction across every trial. The contribution of Optimized_7D is therefore better characterized by its cross-architecture generalization (Section 4.6) and 46% dimensionality reduction than by marginal per-seed accuracy differences.

To provide a more granular view of classification behavior, Figure 2 presents per-class confusion matrices for all three feature configurations. AdDisplay and Adware show consistent recall improvements from LDP_5D to Optimized_7D (+1.6% and +1.8%, respectively), suggesting that global connectivity features, such as reciprocity and reachability, provide discriminative structural cues for these categories. Downloader achieves near-perfect recall (99.5%) across all configurations, indicating that its FCG structure is highly distinctive regardless of feature set. Trojan recall recovers from 89.8% (Full_13D) to 90.8% (Optimized_7D), consistent with the noise-removal effect observed in the overall accuracy analysis.

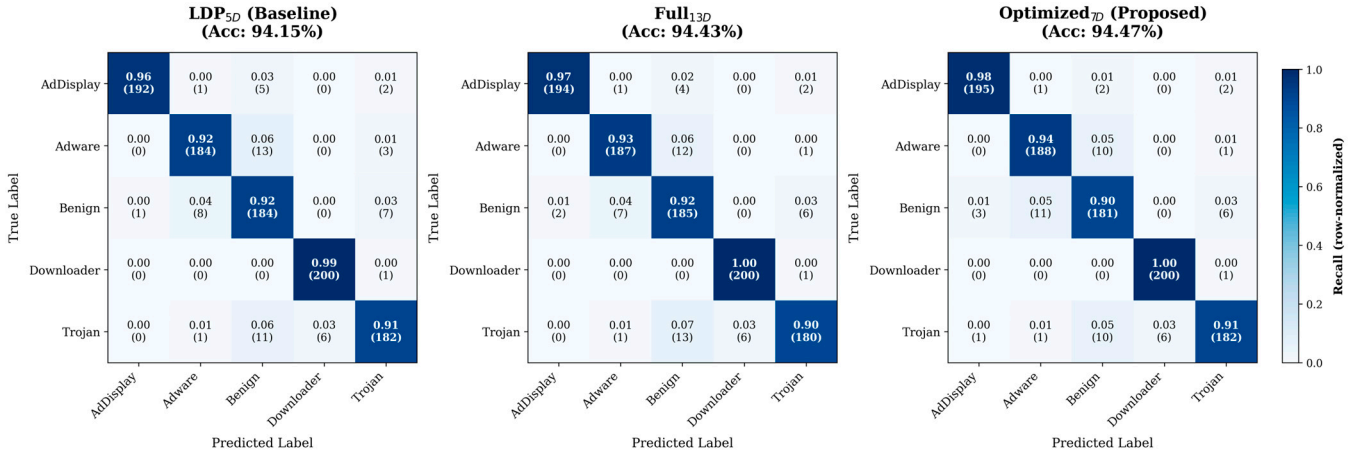


Figure 2. Confusion matrices of JK-GraphSAGE across three feature configurations.

Notably, Benign recall decreases from 92.2% (LDP_5D) to 90.3% (Optimized_7D), with misclassifications primarily directed toward Adware and Trojan. This decline suggests that the removed features (std_neighbor, max_neighbor, clustering_asymmetry) may capture

local structural patterns that help distinguish benign applications from structurally similar malware categories. While this trade-off results in a slight reduction in Benign-specific recall, the overall macro-averaged accuracy improves due to gains in other classes, and the absolute Benign recall remains above 90% across all seeds.

Beyond classification accuracy, the 46% reduction in feature dimensionality (13→7) provides practical benefits for deployment. The reduced feature matrix size (from 13 KB to 7 KB for a typical 1000-node FCG) enables more efficient batch processing and lower GPU memory consumption during training. Additionally, the preprocessing time for feature extraction decreases proportionally, facilitating a faster analysis in large-scale malware detection pipelines. These computational savings become increasingly significant when scaling to datasets like the full MalNet with millions of graphs.

4.6. Generalization Performance Across Architectures

While Section 4.5 confirmed the effectiveness of Optimized_7D on JK-GraphSAGE, this section verifies whether the proposed feature set is generally effective across other GNN architectures, not specialized to JK-GraphSAGE alone. We evaluate LDP_5D, Full_13D, and Optimized_7D on four models: GCN, GIN, GraphSAGE, and JK-GraphSAGE. Table 10 summarizes the test accuracy and improvement margins (Δ) relative to LDP_5D and Full_13D for 12 combinations (four architectures × three feature sets).

Table 10. Architecture-wise Feature Set Performance Comparison.

Architecture	LDP_5D (%)	Full_13D (%)	Optimized_7D (%)	Δ(5D→7D)	Δ(13D→7D)
GCN	89.85 ± 0.42	92.53 ± 0.27	92.61 ± 0.51	+2.76	+0.08
GIN	93.13 ± 0.49	92.93 ± 0.53	93.25 ± 0.40	+0.12	+0.32
GraphSAGE	93.73 ± 0.61	94.19 ± 0.38	94.07 ± 0.34	+0.34	−0.12
JK-GraphSAGE	94.15 ± 0.11	94.43 ± 0.23	94.47 ± 0.25	+0.32	+0.04

Optimized_7D improves accuracy over LDP_5D across all four architectures, showing the largest absolute improvement in GCN and achieving performance gains of approximately 0.1–0.3% in GIN, GraphSAGE, and JK-GraphSAGE. Compared to Full_13D, Optimized_7D records slightly higher accuracy in GCN, GIN, and JK-GraphSAGE, while showing a slight decrease in GraphSAGE but still maintaining higher accuracy than LDP_5D.

By architecture, GCN has the lowest baseline performance and weak directional reflection due to its mean aggregation structure, thus benefiting most from dual-level structural features and selected global features. While GIN showed performance degradation in Full_13D due to noise from some global features being amplified through sum aggregation, in Optimized_7D these features are removed, improving both performance and standard deviation. GraphSAGE and JK-GraphSAGE provide small additional improvements from already high baselines, showing that the selected seven features operate stably in sampling-based aggregation and multi-scale integration structures.

These results support that the features selected using JK-GraphSAGE demonstrate generalization across architectures and provide simpler yet more robust models across various GNN backbones with different message passing mechanisms, such as mean, sum, sampling, and JK.

4.7. Computational Overhead Analysis

To assess the computational feasibility of the proposed framework, we measured the per-graph processing time across 1,000 test graphs from MalNet-Tiny (graph size range: 3–4817 nodes). As shown in Table 11, feature preprocessing required a median of 22.82 ms per graph, while model inference required a median of 3.14 ms. The high standard deviation observed in preprocessing (±78.64 ms) reflects graph size variance rather than algorithmic

instability, as confirmed by the size-stratified analysis: small graphs (≤113 nodes) required only 1.01 ms, whereas large graphs (>1581 nodes) required 110.51 ms.

Table 11. Feature Extraction and Inference Time per Graph (MalNet-Tiny Test set, n = 1000).

Stage	Mean ± Std (ms)	Median (ms)
Preprocessing (offline, one-time)	54.17 ± 78.64	22.82
LDP extraction (5 features)	16.58 ± 22.49	10.87
Global extraction (8 features)	37.59 ± 63.81	11.45
-SCC computation	3.13 ± 19.08	1.52
-K-hop BFS	8.01 ± 23.42	2.64
-khop_density + reachability	22.15 ± 40.68	5.41
-Call depth BFS	0.69 ± 0.74	0.46
-Clustering asymmetry	2.06 ± 2.32	1.14
-fan_out/reciprocity/is_entry	1.55 ± 12.61	0.59
Model inference (online)	3.60 ± 2.16	3.14

Importantly, feature preprocessing is a one-time offline cost performed prior to deployment. In a real-time detection scenario, only the inference step (median 3.14 ms per graph) contributes to latency. Among global features, the k-hop BFS required a median of 2.64 ms, and khop_density and reachability computation required 5.41 ms, confirming that even the most computationally demanding components remain practical for deployment.

5. Discussion and Conclusions

In Section 4, we validated that the proposed dual-level feature framework and interpretability-based selection technique demonstrate quantitatively superior performance across various GNN models. This section discusses the implications of these experimental results, summarizes the final conclusions of this study, and outlines future research directions. In particular, we examine how the quality of input features interacts with the GNN aggregation mechanisms, then comprehensively present the core contributions of this research.

5.1. Discussion: Impact of Feature Quality on GNN Aggregation

The experimental results of this study go beyond demonstrating the performance superiority of the proposed feature set, providing important insights into how the quality of input representations interacts with model aggregation mechanisms in GNN-based malware detection.

The experimental results reveal that aggregation mechanisms critically influence how features contribute to classification performance. Mean aggregation (GCN, GraphSAGE, JK-GraphSAGE) distributes neighbor information evenly, making it relatively robust to noise. As a result, performance improvements are maintained with Full_13D (+0.28~+2.68%), and additional gains from Optimized_7D are modest. By contrast, sum aggregation (GIN) equally amplifies all features, including noise, leading to a performance decrease with Full_13D (−0.20%) when weak discriminative features are present. However, GIN substantially improves (+0.32%) with Optimized_7D through noise removal, demonstrating that feature selection is essential for sum aggregation-based models.

This pattern extends to feature effectiveness across architectures. Full_13D provides performance improvements for some architectures through global structural information, but not all global features are universally effective. Conversely, Optimized_7D selected through interpretability reduces dimensionality from 13 to 7 while simultaneously improving performance and stability, showing consistent improvement trends across all tested

architectures. This demonstrates that the proposed feature set provides noise-robust and generalizable structural representations independent of specific model designs.

First, the effect of feature selection varies according to the structural characteristics of the models. GCN and GraphSAGE, which use mean aggregation functions, showed relatively small performance degradation even when using the noise-containing Full_13D feature set. This is because the averaging operation tends to dilute the influence of outliers and noise features to some extent.

By contrast, the GIN model, which secures high expressive power by summing (sum) information from neighboring nodes, showed a performance decline from 93.13% to 92.93% when applying Full_13D. This can be interpreted as sum aggregation amplifying not only useful information but unnecessary noise. The fact that GIN's performance recovered and improved to 93.25% when applying interpretability-refined Optimized_7D suggests that, for GNNs with strong expressive power, the purity of input features is a major determinant of performance.

Second, interpretability can be utilized as a practical engineering tool for performance optimization beyond being merely a post hoc explanation tool. Unlike the existing studies that primarily use XAI techniques to analyze model internals, this study integrated three techniques—integrated gradients, GradCAM, and SHAP—into the feature engineering pipeline to derive Optimized_7D. As a result, despite approximately 46% dimensionality reduction compared to the LDP, consistent performance improvements were achieved across four GNN architectures. This demonstrates that performing explainable feature selection by quantifying the internal contribution of black box models is an effective strategy for mitigating the curse of dimensionality and securing model generalization performance.

Synthesizing this discussion, we can confirm that, in GNN-based Android malware detection, not only the model-centric approach of designing deeper and more complex models, but the data- or representation-centric approach of designing representations that faithfully reflect the structural characteristics of data should be considered together.

5.1.1. Robustness and Limitations in Static Analysis

As with all static analysis approaches, the proposed framework is subject to the evasion techniques commonly employed by sophisticated malware. Specifically, reflection-based invocation, dynamic class loading, and native code execution can introduce calls that are absent from statically extracted FCGs, causing the graph topology to incompletely represent runtime behavior. These omissions directly affect global structural features: reflection can fragment community structures detected by khop_density, dynamic loading can suppress reciprocity patterns, and native code boundaries can truncate call_depth hierarchies. While FCG-based approaches are generally more robust to syntactic obfuscation than signature-based methods, semantic obfuscation targeting call graph topology remains a limitation. Future work should investigate the resilience of the proposed features under adversarial graph perturbation and hybrid static–dynamic analysis pipelines.

5.1.2. Dataset Scope and Generalizability

The current evaluation is conducted exclusively on MalNet-Tiny, a balanced benchmark of 5000 graphs across five malware categories. While MalNet-Tiny is a widely adopted standard benchmark that enables a direct comparison with prior work, its balanced class distribution and category-level labels may not fully reflect the diversity encountered in real-world deployment scenarios. Validation on the full MalNet dataset (1.2 million graphs, 47 types, 696 families) and additional benchmarks, such as Drebin, represents an important direction for future work to confirm the generalizability of the proposed seven-dimensional feature set across more diverse malware families and graph size distributions.

5.2. Conclusions

This paper proposed an interpretability-driven dual-level feature selection framework to overcome the limitations of input node feature representations in function call graph (FCG)-based Android malware detection. To this end, in addition to the local structural information provided by the existing LDP, we designed eight types of global features reflecting global execution flow and community structure to construct a 13-dimensional structural feature space. Subsequently, to prevent performance degradation due to indiscriminate dimensionality increase, we applied an ensemble rank aggregation strategy combining integrated gradients, GradCAM, and SHAP to select seven optimal features (Optimized_7D) with high contribution to malicious behavior identification.

The experimental results on the MalNet-Tiny benchmark showed that Optimized_7D achieved 94.47% accuracy with the JK-GraphSAGE model, surpassing the LDP-based baseline (94.15%), and demonstrated model-agnostic performance improvements across various GNN architectures including GCN, GIN, GraphSAGE, and JK-GraphSAGE. This suggests that the proposed representation-centric framework can simultaneously improve detection accuracy and reliability without significantly increasing model complexity.

Future research directions include extending the proposed framework to dynamic graph environments to reflect malware behavior patterns that change over time, and further exploring a feature design that enhances adversarial robustness against graph perturbation attacks. Furthermore, important challenges remain, such as instance-level explanation and security analysis support in practical environments, and validation of generalization to the full MalNet dataset and other domains. We expect that the representation-centric interpretable feature engineering approach presented in this study will be utilized as a foundational technology for simultaneously improving the performance and reliability of next-generation intelligent malware detection systems.

Author Contributions: Conceptualization, N.K. and B.L.; methodology, G.K.; software, G.K. and D.Y.; validation, G.K. and D.Y.; formal analysis, G.K. and N.K.; investigation, G.K. and D.Y.; data curation, G.K. and D.Y.; writing—original draft preparation, G.K. and D.Y.; writing—review and editing, N.K. and B.L.; visualization, G.K.; supervision, N.K. and B.L.; project administration, N.K. and B.L.; funding acquisition, B.L. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by an Institute of Information & Communications Technology Planning & Evaluation (IITP)-Innovative Human Resource Development for Local Intellectualization program grant funded by the Korean government (MSIT) (IITP-2026-RS-2022-00156334).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data used in this study are publicly available as part of the MalNet-Tiny dataset, which can be accessed from the official repository.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Zhou, Y.; Jiang, X. Dissecting Android Malware: Characterization and Evolution. In Proceedings of the 2012 IEEE Symposium on Security and Privacy, San Francisco, CA, USA, 20–23 May 2012; pp. 95–109. [CrossRef]
2. Arp, D.; Spreitzenbarth, M.; Hubner, M.; Gascon, H.; Rieck, K. DREBIN: Effective and Explainable Detection of Android Malware in Your Pocket. In Proceedings of the 21st Annual Network and Distributed System Security Symposium (NDSS), San Diego, CA, USA, 23–26 February 2014. [CrossRef]
3. Gandotra, E.; Bansal, D.; Sofat, S. Malware Analysis and Classification: A Survey. *J. Inf. Secur.* **2014**, *5*, 56–64. [CrossRef]
4. Kipf, T.N.; Welling, M. Semi-Supervised Classification with Graph Convolutional Networks. In Proceedings of the 5th International Conference on Learning Representations (ICLR), Toulon, France, 24–26 April 2017. [CrossRef]

5. Hamilton, W.L.; Ying, R.; Leskovec, J. Inductive Representation Learning on Large Graphs. In Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, 4–9 December 2017; pp. 1025–1035. [CrossRef]
6. Xu, K.; Hu, W.; Leskovec, J.; Jegelka, S. How Powerful are Graph Neural Networks? In Proceedings of the 7th International Conference on Learning Representations (ICLR), New Orleans, LA, USA, 6–9 May 2019. [CrossRef]
7. Xu, K.; Li, C.; Tian, Y.; Sonobe, T.; Kawarabayashi, K.; Jegelka, S. Representation Learning on Graphs with Jumping Knowledge Networks. In Proceedings of the 35th International Conference on Machine Learning (ICML), Stockholm, Sweden, 10–15 July 2018; pp. 5453–5462. [CrossRef]
8. Freitas, S.; Dong, Y.; Neil, J.; Chau, D.H. A Large-Scale Database for Graph Representation Learning. In Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, Virtual, 6–14 December 2021; Available online: https://openreview.net/forum?id=1xDTDk3XPW (accessed on 8 March 2026).
9. Lo, W.W.; Layeghy, S.; Sarhan, M.; Gallagher, M.; Portmann, M. Graph Neural Network-based Android Malware Classification with Jumping Knowledge. In Proceedings of the 2022 IEEE Conference on Dependable and Secure Computing (DSC), Edinburgh, UK, 22–24 June 2022; pp. 1–9. [CrossRef]
10. Niu, W.; Cao, R.; Zhang, X.; Ding, K.; Zhang, K.; Li, T. OpCode-level Function Call Graph Based Android Malware Classification Using Deep Learning. *Sensors* **2020**, *20*, 3645. [CrossRef] [PubMed]
11. Gao, H.; Cheng, S.; Zhang, W. GDroid: Android Malware Detection and Classification with Graph Convolutional Network. *Comput. Secur.* **2021**, *106*, 102264. [CrossRef]
12. Bengio, Y.; Courville, A.; Vincent, P. Representation Learning: A Review and New Perspectives. *IEEE Trans. Pattern Anal. Mach. Intell.* **2013**, *35*, 1798–1828. [CrossRef] [PubMed]
13. Cai, H.; Zheng, V.W.; Chang, K.C. A Comprehensive Survey of Graph Embedding: Problems, Techniques, and Applications. *IEEE Trans. Knowl. Data Eng.* **2018**, *30*, 1616–1637. [CrossRef]
14. Sundararajan, M.; Taly, A.; Yan, Q. Axiomatic Attribution for Deep Networks. In Proceedings of the 34th International Conference on Machine Learning (ICML), Sydney, Australia, 6–11 August 2017; pp. 3319–3328. [CrossRef]
15. Selvaraju, R.R.; Cogswell, M.; Das, A.; Vedantam, R.; Parikh, D.; Batra, D. Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), Venice, Italy, 22–29 October 2017; pp. 618–626. [CrossRef]
16. Lundberg, S.M.; Lee, S.I. A Unified Approach to Interpreting Model Predictions. In Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, 4–9 December 2017; pp. 4768–4777. [CrossRef]
17. Shokouhinejad, H.; Razavi-Far, R.; Higgins, G.; Ghorbani, A.A. Enhancing GNN Explanations for Malware Detection with Dual Subgraph Matching. *Mach. Learn. Knowl. Extr.* **2025**, *8*, 2. [CrossRef]
18. Yang, H.; Zhou, Y.; Ji, X.; Liu, Z.; Tian, Z.; Tang, Q.; Shi, Y. Advancing Graph Neural Networks for Complex Relational Learning: A Multi-Scale Heterogeneity-Aware Framework with Adversarial Robustness and Interpretable Analysis. *Mathematics* **2025**, *13*, 2956. [CrossRef]
19. Damodaran, A.; Di Troia, F.; Visaggio, C.A.; Austin, T.H.; Stamp, M. A Comparison of Static, Dynamic, and Hybrid Analysis for Malware Detection. *J. Comput. Virol. Hack. Tech.* **2017**, *13*, 1–12. [CrossRef]
20. Blasing, T.; Batyuk, L.; Schmidt, A.D.; Camtepe, S.A.; Albayrak, S. An Android Application Sandbox System for Suspicious Software Detection. In Proceedings of the 5th International Conference on Malicious and Unwanted Software (MALWARE), Nancy, France, 19–20 October 2010; pp. 55–62. [CrossRef]
21. Cai, L.; Wang, Y. A Simple Yet Effective Baseline for Non-Attributed Graph Classification. *arXiv* **2018**, arXiv:1811.03508. [CrossRef]
22. Newman, M.E.J.; Girvan, M. Finding and Evaluating Community Structure in Networks. *Phys. Rev. E* **2004**, *69*, 026113. [CrossRef] [PubMed]
23. Blondel, V.D.; Guillaume, J.L.; Lambiotte, R.; Lefebvre, E. Fast Unfolding of Communities in Large Networks. *J. Stat. Mech. Theory Exp.* **2008**, *2008*, P10008. [CrossRef]
24. Fan, M.; Liu, J.; Luo, X.; Chen, K.; Tian, Z.; Zheng, Q.; Liu, T. Android Malware Familial Classification and Representative Sample Selection via Frequent Subgraph Analysis. *IEEE Trans. Inf. Forensics Secur.* **2018**, *13*, 1890–1905. [CrossRef]
25. Garcia, J.; Hammad, M.; Malek, S. Lightweight, Obfuscation-Resilient Detection and Family Identification of Android Malware. *ACM Trans. Softw. Eng. Methodol.* **2018**, *26*, 11. [CrossRef]
26. Tarjan, R. Depth-First Search and Linear Graph Algorithms. *SIAM J. Comput.* **1972**, *1*, 146–160. [CrossRef]
27. Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; et al. Scikit-learn: Machine Learning in Python. *J. Mach. Learn. Res.* **2011**, *12*, 2825–2830.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.